

InsightEngine

Building a Market Intelligence Platform, Start to Finish

Prototyping Products with Data and Artificial Intelligence • April 2026

This is the story of InsightEngine: a market intelligence platform we built over a semester that turns raw customer reviews, stock prices, and news signals into audience-specific competitive dashboards. The idea started simple. The execution did not. This post documents the journey honestly—what worked, what we threw away, and what we learned about building with LLMs under real constraints.

The Problem We Wanted to Solve

The question that started everything was almost embarrassingly basic: why does competitive analysis still take weeks? The raw material—millions of customer reviews, news headlines, earnings reports—is sitting in public view. The reasoning capacity to synthesise it exists in modern language models. The bottleneck, we decided, was not data or compute. It was *framing*.

A stock analyst evaluating HubSpot and a product manager at a competing CRM look at the same review corpus but need completely different things. The analyst wants sentiment trajectory and share-of-voice signals over time. The product manager wants a ranked list of the features customers complain about most. A buyer choosing between tools needs a clean value-for-money comparison. Most BI tools are built for one of these audiences and serve the other two poorly. We wanted to serve all three from a single data run.

The first sketch had three boxes on a whiteboard: *scrape reviews*, *score with LLM*, *show chart*. It took about one afternoon of actual implementation for the diagram to become considerably more complex.

What We Built

InsightEngine is a web application where a user logs in, picks an audience (Investor, Company, or Customer) and one of twelve industry verticals—CRM, food delivery, ride-sharing, SaaS collaboration, e-commerce, restaurants, hospitality, banking, healthcare, travel, telecom, and insurance—then sees a dashboard of charts and AI-generated insights tailored to that combination. The same underlying data produces three entirely different views depending on who is asking.

The platform covers between three and six competing companies per industry, scored across five to eight competitive dimensions, producing a normalised score matrix that drives all the visualisations. Before reaching the dashboard, users pass through a short onboarding chat where they describe their role and what they are trying to understand. The LLM extracts structured preferences from that conversation and surfaces a personalised insight section above the standard analysis.

On the technical side, a React frontend talks to a FastAPI backend over a REST and SSE API. The backend pulls review data from up to four sources, scores each review individually with a language model, synthesises insights in a single generation call, then enriches the result with live stock prices, news headlines, and Glassdoor ratings. The whole pipeline runs in under thirty seconds on Groq’s free tier.

Layer	Technology
Frontend	React 18, Vite 6, TypeScript, Tailwind CSS v4, Recharts
Backend	Python 3.9, FastAPI, Uvicorn
Database	SQLite via SQLAlchemy (auth, saved views, chat history)
LLM	Groq (11ama-3.3-70b) or OpenAI, user-selectable
Reviews	Apify (G2, Capterra), SerpAPI Google Maps, Kaggle datasets
Market	Alpha Vantage (stock quotes), SerpAPI (News, Trends, Glassdoor)
Auth	JWT via python-jose, bcrypt password hashing

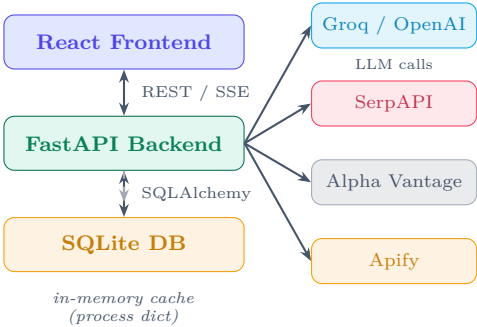


Figure 1: High-level architecture. The React frontend communicates with FastAPI over REST and SSE. The backend calls external APIs for LLM inference, market data, and review scraping, and persists auth data in SQLite.

How the Architecture Actually Came Together

The three-box whiteboard diagram became a four-stage pipeline fairly quickly. The **Fetch** stage sources reviews in priority order—Kaggle datasets first (largest and most structured for supported industries), then SerpAPI Google Maps for restaurant and hospitality verticals, then Apify’s managed scrapers for G2 and Capterra, and finally a local JSON file as a final fallback that is always available. The **Score** stage submits each review to a fast language model—**11ama-3.1-8b-instant**—with a structured prompt, running up to eight reviews concurrently. The **Insights** stage sends the full score matrix and a selection of representative excerpts to a larger model—**11ama-3.3-70b-versatile**—in a single synthesis call, getting back both a JSON array of tagged insights and a short *executive brief*: a plain-prose paragraph that orients the reader before they encounter individual insight cards. The **Enrich** stage fetches live signals independently: stock price history, PE and PEG ratios, news headlines, Google Trends share-of-voice, and Glassdoor ratings.

One architectural decision that shaped everything else was running a single pipeline and fanning out the results to three audience views rather than running three separate pipelines. The practical reason was simple: Groq’s free tier has token-per-minute limits, and tripling the number of synthesis calls would have pushed a full analysis run close to four minutes while risking rate limit errors. Collapsing to one pipeline

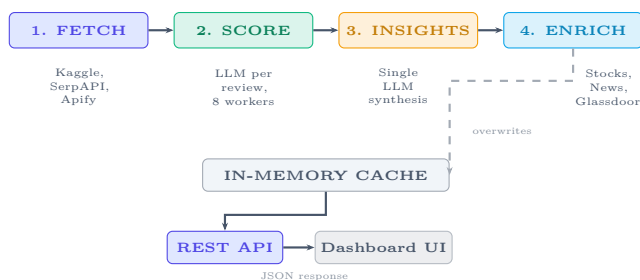


Figure 2: Four-stage SSE pipeline. Each stage emits a progress event to the frontend. Results overwrite the in-memory cache and serve all subsequent page loads.

forced us to design the insight generation prompt so the LLM tags each insight with an audience label—Investors, Companies, or Customers—at generation time. The frontend then applies a second gate via a `CHARTS_BY_AUDIENCE` map, ensuring each tab renders only the charts and data cards relevant to that stakeholder type. What started as a cost optimisation became the platform’s most distinctive feature.

The audience routing model ended up looking like Figure 3. Each tab sees a completely different set of charts: investors get stock performance history with five-year price charts, PE and PEG ratios, news headlines, sentiment trends, and share-of-voice; companies get the radar chart, a full dimension heatmap, score deltas versus the category benchmark, and Glassdoor employee ratings; customers get a head-to-head comparison table with the per-dimension winner highlighted, AI-matched recommendation cards by buyer profile, and a positioning map of price versus perceived value.

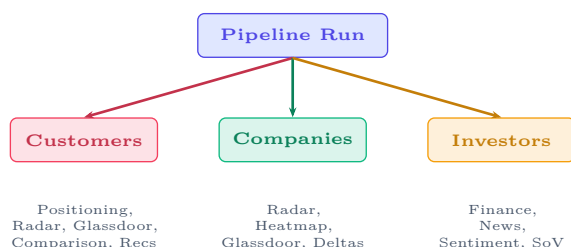


Figure 3: One pipeline run fans out to three independent views, each with its own chart set and insight bucket.

The Parts That Took the Most Time

Getting the LLM to Score Consistently

Scoring reviews with a language model sounds straightforward until you actually do it. Each review is submitted to `llama-3.1-8b-instant` with a prompt defining the task, the list of competitive dimensions, and an instruction to return a JSON object with one integer score per dimension from 0 to 100. The model should return null for dimensions not mentioned in the review.

In practice, early versions of the prompt produced a mean score of 71 across all dimensions. Every company looked good and the comparative signal was buried in a narrow band at the top of the range. The model was not lying—71 is a plausible reading of most customer reviews, which tend toward the positive unless something went seriously wrong. The problem was that a compressed scale is useless

for comparison. We tried lowering the anchor, rewording the task, and adding explicit boundary examples. What finally worked was specifying in the prompt that a score of 50 represents a genuinely mixed or neutral experience, and that scores below 40 should be reserved for reviews that describe concrete failures. Adding those anchors pushed the distribution into a range where differences between companies were actually visible.

The same synthesis call that produces the insight array also generates the executive brief—a two to three sentence paragraph in plain prose that summarises the key competitive dynamic for the current audience. This was added after early user testing: without it, readers would land on a list of individual insight cards with no orienting context. The brief is toggleable so returning users can skip it.

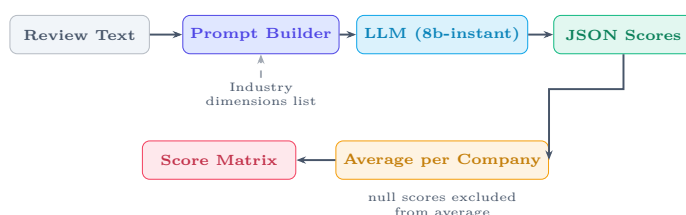


Figure 4: Per-review LLM scoring flow. Up to eight reviews run concurrently. Dimensions not mentioned in a review return null and are excluded from the per-company average.

Scraping turned out not to be one thing

What the whiteboard diagram called “scrape reviews” became four separate data paths. G2 and Capterra require a managed scraper that handles login walls and pagination—we used Apify for that. Google Maps reviews have a completely different structure and are more representative than software review platforms for restaurants and hospitality, so SerpAPI’s Maps endpoint handles those. For food delivery and ride-sharing, Kaggle hosts pre-built datasets that are richer than anything we could scrape live. And beneath all of that sits a set of local JSON files with sample reviews that always work regardless of which API keys are present.

Wrapping all of this in a graceful fallback chain—try Kaggle, then SerpAPI, then Apify, then local file—was one of the cleaner decisions we made. The `resources/adapters.py` file acts as a central dispatcher that catches import errors and missing keys and returns `None` rather than crashing. The calling code checks for `None` before including a field in the API response, and the frontend renders a card only when the corresponding data field is non-null. The platform demos at any point in development because the pipeline degrades gracefully instead of failing visibly.

What We Threw Away

The first version of the codebase ran three separate analysis pipelines, one per audience. It felt natural to model it that way. It was also a disaster: total run time crept past four minutes, Groq rate limits became unreliable, and the three synthesis prompts produced subtly inconsistent framing. Collapsing to a single pipeline with audience fan-out cut run time by roughly 60% and produced a design that was actually better—one where the insight routing decision is made by the LLM during generation rather than by keyword matching after the fact.

The original frontend was a single scrollable page with

all charts visible and an audience dropdown. On a real screen this was overwhelming: a radar chart, a competitive heatmap, a stock chart, and a news feed all loaded simultaneously with no clear reading order. Switching to a tabbed interface with one tab per audience took half a day of work and made the product noticeably more convincing in a demo context. URL routing was retained so that each tab has its own bookmarkable address.

We also planned to use the OpenWebNinja Glassdoor API for employee ratings. The key was added to the environment file and we tried to connect to the endpoint, but the API returned an AWS API Gateway “Missing Authentication Token” response regardless of what path we attempted, and the documentation was too sparse to debug systematically. Rather than spend days on a third-party API with no clear path forward, we pivoted to extracting ratings from Google Search snippets via SerpAPI—a targeted `site:glassdoor.com` search whose results are parsed with a regular expression. It works about 70% of the time, which was acceptable under time constraints.

An earlier version of the Investors tab included a customer migration flow visualisation, showing movement between competitors based on hash-derived values. During review we realised that entirely synthetic data presented in a chart format gives the impression of real analytical validity, when in fact it had none. We removed it rather than leave something misleading in the product.

What We Learned About Building with AI

Claude Code was the primary coding assistant throughout. For tasks with clear specifications—the SSE streaming loop, the SQLAlchemy models, the JWT auth flow, the Recharts visualisation components—AI-generated code was correct on the first attempt and needed only minor adjustments. For tasks requiring accumulated project context, such as the audience routing logic or the cache fallback strategy, the AI needed explicit framing before it could produce something architecturally coherent. The adapter pattern in `resources/adapters.py`—nested `try/except ImportError` blocks around every optional dependency—was suggested by Claude Code when we asked how to make optional packages safe. We would have arrived at something similar, but the AI applied it consistently across all adapters immediately, which saved time and produced a more uniform codebase.

LLM prompts required more iteration than we expected. Initial drafts for both the review scoring and the insight synthesis prompts went through roughly ten rounds of refinement before the outputs were consistent enough to trust. AI-assisted prompt engineering—generating a sample output, identifying a failure mode, asking the model to suggest a fix—reduced the number of manual iterations considerably.

Several bugs were diagnosed with AI help. The most significant was a race condition in the chat streaming endpoint where a database session was being shared across concurrent SSE events, leading to intermittent SQLAlchemy errors. The fix—moving session creation inside the generator function—was identified correctly from the error message alone. A second bug was a `NoneType` division error when the LLM returned null for a dimension used in a position-

ing calculation. The fix was a one-line change suggested immediately upon showing the traceback.

AI assistance had real limits. For anything requiring understanding of *why* a specific workaround was in place—a quirk of the SerpAPI Glassdoor extraction, or the cache invalidation order—the AI required substantial context and sometimes produced suggestions that were locally correct but architecturally misaligned. Every AI-generated code change needed human review. The AI also occasionally introduced subtle issues, such as replacing straight quotes with typographic curly quotes in source files, which caused build failures.

The broader lesson is that AI tooling changed the shape of development work more than it changed the amount. Configuration, boilerplate, and debugging took less time. Architectural decisions, prompt engineering, and verifying that generated code actually matched the design intent took at least as much as they would have without AI assistance.

Where It Landed

The final platform covers twelve industries, scores up to 64 reviews per run across six companies in under thirty seconds, and delivers three meaningfully different dashboards from a single data pipeline. Investor and company views diverge substantially on the same underlying dataset—confirming that audience routing produces distinct signals rather than merely relabelling the same content. The live enrichment layer—real stock prices, valuation tables, news headlines, and Glassdoor ratings on the Investors tab—creates a noticeably more credible experience than a purely review-driven view.

Some constraints remain. All analysis results are lost on server restart because the cache lives in process memory rather than a database. Alpha Vantage’s free tier allows 25 requests per day, and the stock chart integration now makes three calls per company (quote, monthly series, and company overview), so a single run across six publicly traded companies consumes 18 of those 25 daily requests. The SSE stream holds a Uvicorn worker thread for its full duration, which means concurrent users compete for the same thread pool—acceptable for a prototype, not for a production deployment. The OpenWebNinja Glassdoor integration was never completed; SerpAPI remains the sole source.

If we had more time, the highest-value additions would be persisting analysis results across restarts, replacing the in-process pipeline with a background task queue, and deepening the onboarding chat so it shapes not just the insight highlights but the chart selection and framing as well. The audience-routing model is the platform’s most interesting axis for future development. The insight that different stakeholders need different framings of the same data seems obvious in retrospect, but it is genuinely underserved by existing tools—and it ended up being the thing that made every demo more interesting than we expected.